

How the dashboard was built

This is the companion to `dashboard_handbook.md`. The handbook explains how to use the dashboard. This one explains how it came to be the way it is: what we set out to build, what we found when we tested it, what we had to change, and what we can actually prove about it.

It is written mostly from two records. `reports/integration-design.md` is the original plan, written before any of this existed. `dashboard/FILTER_SPEC.md` is the implementation and decision log, and it is the more valuable of the two because it contains the arguments we lost as well as the ones we won. The architecture history and the engine benchmark in section 2 are not in either file and come from the working record instead, which is flagged again at the end. Where a decision is recorded I have kept the original reasoning. Where something is not written down anywhere I have said so rather than reconstructing a plausible story.

The short version: almost nothing about the final architecture matches the original plan, and the two most important corrections in the project came from checking work we had already called finished.

1. The problem, as it actually was

The original design document states it in one line:

Nobody can answer "what do we know about company 12345678" without opening six notebooks.

That was the whole problem. Eight branches, roughly twenty notebooks, each producing a different shape of output, saved in a different place, with a different idea of what a company key looks like. The document is explicit that this was the *only* problem being solved:

That is the only problem we are solving. We are **not** rebuilding anyone's pipeline.

That constraint shaped everything downstream. Every source in the dashboard today is still owned by the notebook that produced it, and nothing upstream was rewritten to suit the dashboard.

2. What we planned, and why it was wrong

The architecture went through three versions before it settled, and the second one is the interesting one, because it was fully designed and was still the wrong answer.

The first plan: a signals table

The plan was a **signals table**. Every pipeline would keep working as it did, and each would get a thin adapter, about thirty lines, that translated its output into rows of one shared shape:

```
your notebook  -> adapter  -> signals table  -> dashboard
(unchanged)      (~30 lines) (one shared      (reads only,
                                shape)          never fetches)
```

One row per company per fact per date, with `company_number`, `signal_type`, `signal_date`, `value`, `detail`, `source`, `confidence` and `retrieved_at`. All of it precomputed into a **store**, a finished database file the dashboard would read.

Three rules came with it, and these did survive, all three:

1. Always join on company number, never on company name.
2. Every signal must have a real date, no nulls.
3. **A company with no signals is a real answer, not a missing one.**

The third is the single most durable idea in the project. It is why the source tiles distinguish "none found" from "not searched", why zero news coverage is presented as a finding, and why the change panel says how many of its checks have no history to compare against.

The second plan: ship the data to the browser, in pieces

The signals table was going to be delivered to the browser as JavaScript, and the first attempt at that was one file. `data.js` grew to 27 MB holding only part of what was needed; the full set across all sources came to somewhere between **85 and 100 MB**. No one is going to wait for that.

So the design was sharded, and it was a genuinely reasonable design:

```
1.53M company universe  (parquet, unchanged)
    |
    core.js ~5 MB · loaded once at startup
    105,078 signalled companies: number, name, status, sector,
    segment, and a bitmask of which sources each one has
    |
    +-----+-----+
    v               v
features/<src>/NN.js  events/<src>/NN.js
~150 KB/shard        ~320 KB/shard
```

The sharding key was the **last two characters of the company number**, giving 100 shards per source per kind. Search only ever needs `core.js`, because search is over identity, not evidence. The moment you

open a company you know its number, so you know its shard, and the browser pulls that one file in by injecting a `<script>` tag. Once pulled it stays cached, so the second company you open from the same shard costs nothing.

The arithmetic worked. **About 5 MB up front instead of 85 to 100 MB**, then a few hundred KB when you actually open something. The bitmask in `core.js` meant the interface knew which source tiles to show before it had fetched any of them.

Why it still was not right. Everything in that diagram is precomputed. The 105,078 figure is a build artefact: it is the set of companies that had a signal *at the time the shards were generated*. Any count on screen is only as true as the last build, and the browser holds a second copy of the data that can drift from the parquet. Filtering the full 1.53M was impossible, because the browser only ever had the 105,078 with signals. And every new filter meant regenerating 100 shards.

What we actually built: query in place

`FILTER_SPEC.md` records the architecture inverting underneath the specification:

The architecture changed underneath the spec: the browser is no longer where the company universe lives. DuckDB reads the parquet in place and answers over all 1,531,094 companies, and the API returns only the rows asked for. Nothing is precomputed into a store, no database file is created, and the parquet is never written to.

The consequence line matters more than the change: *"This is why every count in this document is now measurable at runtime rather than baked at build time."* The dashboard cannot disagree with the data, because there is no second copy to disagree with. And the universe went from 105,078 to 1,531,094, because a company with no signals stopped being something you had to precompute a row for.

The residue is still visible. `data.js` is 27 MB on disk that the browser no longer loads at all; the server keeps it only to lift a 20-key metadata block at startup. It is recorded as open item 6, and the fix is to move that block to a small config file.

Choosing the engine: DuckDB against PyArrow

Both read parquet, so this was a query-engine comparison rather than a format one. Measured on this dataset:

Operation	PyArrow	DuckDB	Ratio
Startup	~2 s	24 ms	83×
Filtering	~28 ms	25 ms	1.1×
Name search	~65 ms	52 ms	1.25×
Sorting	370 ms	30 ms	12×
Aggregation	151 ms	17 ms	9×
Memory	444 MB	0.2 MB	2,200×

The two rows people expect to be decisive, filtering and name search, are close to a tie. Had those been the only operations that mattered, either engine would have done. The decision came from the other four, and each maps onto something the dashboard actually does:

- **Aggregation, 9×**. This is the one that settled it. Every facet repaint fires 57 aggregate queries, because the counts beside each filter option have to be live (section 7). A nine-fold penalty on the single most repeated operation in the application is not something you optimise around later.
- **Sorting, 12×**. Every result list is ordered, and after defect B1 (section 9) every one of them carries a tie-break on a second column to guarantee a total ordering. Cheap sorting is what makes correct sorting affordable.
- **Memory, 444 MB against 0.2 MB**. PyArrow materialises the table into memory; DuckDB streams from the parquet and holds almost nothing. This is the difference between a process you have to think about and one you do not, and it is why there is no cache anywhere in the server.
- **Startup, ~2 s against 24 ms**. Least important analytically, most important in practice, because a development server gets restarted constantly.

The through-line from all three architectures is the same: the parquet is the truth, and the further a design moves a copy of it away from that file, the more ways it finds to be wrong.

3. Which company list is the master list

The original document flagged this as the first thing to settle, before any code:

Which company list is the master list? There are currently three, and they disagree: Sam's spine 869,043 · the SHAP feature matrix 1,372,321 · the 33-month panel 2,038,130. A company can be in one and missing from another, so until we pick one, every join quietly drops rows and nobody notices.

The dashboard settled on **1,531,094**, which is none of the three. It is the row count of `dashboard_bulk_gazette_2026-07.parquet` , the July 2026 Companies House bulk file with the Gazette enrichment already joined on.

The decision that matters is not which number won but that **the universe is never narrowed**. The "All Companies" view applies no predicate at all. From `FILTER_SPEC.md` section 1:

All Companies applies no predicate — the full universe, nothing removed.

Everything else is a filter over that, which means a count on screen is always a count out of 1,531,094 and never out of some earlier subset nobody remembers choosing. It also means the answer to "why isn't my company here?" is almost always "it is, you have a filter on".

4. The spine and the side tables

The final shape is one parquet plus six auxiliary files, all read in place:

Layer	What it is	Rows
The spine	<code>dashboard_bulk_gazette_2026-07.parquet</code>	1,531,094 × 124 columns
Gazette notices	<code>nb10_gazette_notices_thru_2026-07.csv</code>	291,045
News	<code>nb14_news_signals_...csv</code> plus <code>news_coverage_summary.csv</code>	96 + 398
Property	<code>land_registry_company_features.csv</code> and its events file	60,629
Trade marks	<code>ipo_trademarks_company_features.csv</code> and its events file	15,713
Grants	<code>ukri_grants_company_features.csv</code>	10,254
Identity	the crosswalk	1,496,693

The split is deliberate and it decides what a source can do. **Anything on the spine can be filtered. Anything in a side table can only be shown on a profile.** Borrowing, lender, filing, momentum and the model scores are columns on the spine, so they are filterable. Property, trade marks and grants are side tables keyed on company number, so they appear on a company page and nowhere else.

That is why the filter panel has no "has property" control, and it is not an oversight. It also produced the one preset the system cannot take apart, which section 9 covers.

5. Company numbers

The original document listed this under "things we already know (bad)":

Company numbers are formatted differently in different places. Some code keeps them exactly as they came, some pads them to 8 digits. Some CSV column names have a leading space. We need one shared cleaning function that everybody calls.

That is what happened. `serve.py` carries a single SQL cleaner, `CLEAN_CN`, applied to every side table as it loads, and its comment records the check:

Canonical company number, matching the rest of the pipeline. Verified row-for-row against the Python cleaner in `build_data.py` on 3,027,787 rows, 0 mismatches.

Two cleaners existed, one in Python for the build and one in SQL for the server, and they were checked against each other on three million rows rather than assumed to agree. Empty strings, `NAN` and `NONE` all resolve to NULL rather than to a company called "NAN".

6. The sources, one at a time

Each source arrived with a decision attached, and most of those decisions are about how to present something honestly rather than how to compute it.

Gazette

The only source matched on company number throughout, so a hit is certain rather than probable. It is also the only source that is both filterable and profile-visible, because its flags are columns on the spine.

The interesting decision is temporal. The Companies House snapshot is 1 July 2026; the Gazette enrichment runs to 31 July. `FILTER_SPEC.md` 4.1 records that this is deliberate:

The dashboard intentionally treats Gazette-through-31-July as enrichment on top of the 1 July snapshot, not as a contradiction of it. Companies House status lags Gazette publication, so July notices are the most valuable rows in the file, not contamination.

That is defensible for a dashboard that describes. It would not be defensible for a model, and the record says so, with the fix pre-loaded:

Condition on this decision: if `gaz_*` fields are ever used as features in predictive modelling, those 740 are circular and must be excluded (`gaz_new_in_july_flag = 1`).

740 companies got their first ever notice in July. **656 of them are still "Active" in Companies House and 586 have never been flagged distressed by CH at all**, which is exactly why the July rows are worth having and exactly why a model must not see them.

The Gazette filter is ten named states rather than one toggle. A bare "Gazette" switch was explicitly excluded as ambiguous. The most important of the ten is **"No notice" at 1,511,569**, because without it the *Silent distress* preset could not exist.

News

The most conservative source and the one whose result looks most like a bug. 467 companies searched out of 1,531,094, and **zero** with coverage that survived verification.

The verification is the story. One company, `16392251`, is recorded in the code by hand:

Matched an article about Flying Tiger Copenhagen. "HOLDINGS" is stripped as a legal suffix, leaving the single word TIGER, which appears inside "Flying Tiger"; the corroborating signal was the town LONDON, which appears in most Guardian business articles. Reviewed and rejected.

That is a false positive produced by two reasonable rules interacting: strip legal suffixes so "TIGER HOLDINGS LTD" matches "Tiger Holdings", and corroborate with the registered town. On a one-word name and a London company, both rules fire on an article about a Danish gift shop.

The handling is worth noting because it would have been easier to delete the row. Instead the raw hit is still counted and only the verification is reversed, so the profile can say a hit was found and rejected rather than quietly showing nothing. Fifteen companies are in that state.

Contracts

On the spine as `contracts_won_12m`, but **not offered as a filter**. This is the source of the one permanent exception in the preset system, covered in section 9.

Property

Land Registry titles, matched on company number. The decision here is about a long tail:

Holdings are wildly skewed: median 1 title, 90th percentile 4, 99th 29, maximum 65,556. The long tail is trust and corporate-services firms holding titles for clients rather than on their own balance sheet, so a profile reading "65,556 properties" would say something untrue about the business.

The threshold is 30, the 99th percentile rounded up, which puts 680 of 60,629 companies on the caveated side. The rule the code states is the one that matters: **the true count is always shown and never altered**; above the threshold the page adds an explanation. Nothing is hidden to make the number look sensible.

Trade marks

The weakest of the packs and labelled as such everywhere. Matched on **name plus postcode area only, confidence 0.85 on all 15,713 rows**.

It also has the worst vintage problem in the project, and it is not a lag but a hard stop. The register behind the file is the IPO's **historical register, which ends on 28 January 2018**. The Intellectual Property Office published that free bulk extract on 13 February 2018 and **has never updated it**. So this is not eight-year-old data that will refresh; it is a closed archive. Every number carries the vintage, because otherwise a reader concludes the company stopped filing marks rather than that the source stopped recording them. The code is correspondingly careful about tense:

Status is also as-at-2018: a mark "Registered" then may have lapsed since, so the screen says "registered as at" and never "currently holds".

Grants

UKRI research funding, matched on name and postcode area rather than number, so it carries a confidence and the tile says "name match only".

Hiring, and why it is not here

The original design assigned Adzuna job adverts to Sam as a hiring signal, notebook 07, described as "already built". It is not in the dashboard, and there are now **zero references to hiring anywhere in the code**.

The evidence for what happened is in `data.js`, the earlier build artefact still on disk. It records:

```
"sources_live":    ["gazette", "news", "contracts"]
"sources_pending": ["hiring", "property", "trademark", "grants"]
```

Four sources were pending at that point. Three of them landed. Hiring did not, because no usable data reached the dashboard, and it was removed rather than left as a permanently empty tile promising something that would never arrive. The live server now reports six sources live and none pending.

That is the honest version of a decision that could easily have been dressed up: we cut a source because we did not have the data for it.

7. The filter system

27 filters across six groups: Core, Borrowing, Lender, Filing, Momentum, Signals. The implementation note explains the shape:

One declarative table driving three consumers: the WHERE builder, the facet-count endpoint, and `/api/filters`, which the UI renders itself from. **Defining a filter once is what stops the panel and the query drifting apart.**

The panel is not hand-written HTML. The browser asks the server what filters exist and draws whatever it is told, so a filter cannot appear in the UI without a predicate behind it, and a predicate cannot change without the panel following.

The rule that governs NULLs

This is the most consequential design rule in the filter system:

A missing value is never treated as a negative. Every filter with NULLs offers three states, Yes / No / Not stated, where "No" means *observed false*, not *unknown*.

It exists because the NULL rates are enormous and they mean specific things:

Filter	NULL rate	What NULL actually means
Main lender	94.4%	No outstanding charge. Not "unbanked"
Size tier moved	41.4%	No 12-month history, or no segment to move
Relocated / Industry changed	13.9%	Incorporated less than 12 months ago
Model scores	8.0%	Company is not Active, so it was never scored

Collapsing any of those to "No" would produce a confidently wrong list. A company incorporated eight months ago has not "not relocated"; we cannot say.

Live counts, and why they are mandatory

Every option carries its own count. The specification argues for this with a worked collapse:

Stack	Companies
Active	1,409,284
+ segment Small	374,875
+ outstanding mortgages > 0	53,086
+ former LBG	2,807
+ competitor entered 12m	56
+ won a contract in 12m	0

Without counts, users build empty queries and conclude the tool is broken.

That table was later used as a verification target in its own right, and it reproduces exactly through the fifth step. The sixth cannot be reproduced because contract activity is not a filter, which is the same gap that appears again in section 9.

8. Presets, and the rule that killed two of them

Presets started as saved combinations. The final pass added a test that a preset had to pass:

Redundancy rule adopted: a preset earns its slot only if it retains **under ~80%** of the population of its most restrictive single-filter component. Otherwise it is a filter, not a preset.

Two failed it.

Preset F, "Won public work recently", retained 93.5% of the "Won in 12m" filter. It was a repackaging. The record shows alternatives being tested rather than the preset simply being cut: never borrowed 2,303 (39.2%), no outstanding borrowing 3,234 (55.1%), first ever award 2,160 (36.8%). The replacement was **F2, "Contract winner with no borrowing", 2,303**, and the note says the business logic improved as a side effect: a company that has just won public work and has never borrowed is the one with a genuine new financing need.

Preset B, at 78.2%, was borderline. Adding the trading-segment restriction took it to 8,992 (63.8%).

A third decision is more subtle. Presets A and H are restricted to trading segments, and the reason is a distinction that is easy to miss:

`is_active` is a *status*, `Dormant` is a *segment*, and **89.4% of Dormant companies hold Active status**, so `is_active` alone does not exclude them.

Two presets, C and D, are deliberately left unrestricted, and the reasoning is commercial rather than technical: *"A deteriorating exposure or a lapsed client entering administration is more urgent, not less. Restricting them would hide the cases that matter most."*

9. The adversarial pass, and the five defects it found

On 19 August a full verification pass was run against the live server. The method matters:

Independent DuckDB ground truth **written from column semantics rather than copied from `serve.py`**, with every count check paired with a row-level re-test of each returned company. Roughly 400 assertions.

Ground truth copied from the implementation only proves the implementation agrees with itself.

The filtering logic came through clean. 63 facet counts, 34 filter cases, 9 presets decomposed into 36 conditions, 22 hand-built combinations including 8 deliberate contradictions, and 120 seeded random combinations produced zero count mismatches and zero rows that failed the predicate they were selected by. Every injection attempt returned 0 rows.

Then it found five defects, and none of them were in the filtering.

B1: ordering was not deterministic

The worst of the five, because it was silently losing data.

The Gazette and Former LBG views ordered by non-unique keys with no tie-break, so DuckDB's parallel scan was free to arrange ties differently on each run. The measured consequence:

- Five identical calls to the LBG view returned **five different sets of companies**
- Walking six pages of 100 showed **526 distinct companies in 600 slots, so 74 were never displayed**
- It reached the UI: growing the list from 25 to 50 moved **15 of the 25 visible rows**

A user paging through a list was not seeing 74 companies, and nothing anywhere indicated that.

The fix is one constant, `TIE_BREAK = ", CompanyNumber ASC"` , appended to every ordered query: both view orders, the default name order, the custom sort branch, and the legacy endpoints. `CompanyNumber` is unique across all 1,531,094 rows and never null, so the sort became a total order.

The lesson, and it is the one that generalises: an ordering that is merely correct is not enough. It has to be *total*, or pagination quietly lies.

B3: filters failing open

Unrecognised values were silently dropped, and the failure widened the result set:

Input	What it did
<code>gazette=not_a_state</code>	Dropped the filter, returned all 1,531,094
<code>repayment=bogus</code>	Same
<code>preset=not_a_preset</code>	Same
<code>view=not_a_view</code>	Same
<code>segment=NoSuchSegment</code>	Correctly returned 0

Same user error, opposite outcomes, and the failure widened the result set.

That is the dangerous direction. A typo that returns nothing is obvious. A typo that returns everything looks like a working query. Fixed with a `FilterError` and a 400 handler that names the valid options.

B2 and B4: input handling

A negative `limit` reached DuckDB as `LIMIT -5` and returned HTTP 500, because only offset was floored. And range inputs had the same split personality as B3: `age_min=nan` silently returned 0 while `age_min=abc` silently returned everything. Two kinds of bad input, two silent and opposite outcomes. Both now return 400.

B5: a real value that looked like a null

27 matched companies carry the genuine severity tier `'none'`. The shared text helper `_txt()` treats the literal string "none" as a null marker, which is correct for the CSV side files and wrong here, so those rows rendered as `· 2 notices` with a dangling separator.

The fix is narrow on purpose: a separate `_cat()` used for `gaz_severity_tier` only, plus a label map rendering it "No severity tier". **Stored values and every filter predicate are unchanged.** A presentation bug got a presentation fix.

10. The investigation that overturned our own conclusion

This is the most valuable thing in the record, because it is the project catching itself being wrong about something it had already decided.

How it started

Preset A tested `debt_ratio = 0`. Cross-verifying a proposed change to `Mortgages.NumMortOutstanding = 0` turned up one company's worth of difference, **ANTALIS LIMITED, 01088345**: 416 charges, 2 still outstanding, 414 satisfied. Its true ratio is 0.0048, and `debt_ratio` is rounded to two decimals upstream, so it stores as 0.00. It is the only company in the entire file whose ratio falls in the 0 to 0.005 band.

Preset A moved to `Mortgages.NumMortOutstanding = 0`. 31,003 became 31,002. One company removed, none added.

What the cross-check then uncovered

The file carries **two different "outstanding" measures and they disagree**:

<code>Mortgages.NumMortOutstanding</code>	the Companies House bulk count
<code>n_charges_outstanding</code>	the charge-level lender pipeline

They differ on 2,859 companies, and on 1,300 of those one is zero while the other is not. Every lender field is built on the pipeline column, not the bulk one, which produces the number that started the argument: **996**

companies where the bulk says nothing is outstanding but a named lender is attached.

Inside preset A, a list whose entire promise is "no incumbent to displace", that left **661 companies carrying a named lender and 118 of them current LBG clients.**

The conclusion we reached first, and it was wrong

The record preserves it rather than deleting it, struck through, with a note explaining why it is still there:

The paragraph that stood here was wrong and has been replaced by 9.6. It read the 661 as "661 companies wrongly in a prospecting list, a business error", and recommended moving preset A onto `n_charges_outstanding = 0` .

That reads like an obvious business error. It is not.

What the investigation found

The disagreement is a function of **company age**, not of data freshness:

Company age	Pipeline says outstanding	Bulk disagrees	Rate
under 10y	20,592	21	0.10%
10-20y	30,298	49	0.16%
20-30y	26,988	70	0.26%
30-50y	18,112	442	2.44%
50y+	7,549	701	9.29%

A ninety-fold gradient. The supporting evidence points the same way: disputed LBG relationships carry an average charge age of **34.4 years** against 12.9 where the two sources agree, **88 of 173 have no satisfaction record at all**, and of the 118 current LBG clients inside preset A, 86 hold a charge over 30 years old while only **4 hold one under 10 years**. The 17 companies disagreeing in the opposite direction show no age gradient and are noise.

That is the signature of **charges left open on the register because a satisfaction was never filed**, not of live secured borrowing. The pipeline's gate holds a charge open whenever the satisfaction date is missing, which is precisely the condition expected on a 1980s charge.

Two checks were run to rule out alternative explanations. It is not a jurisdictional artefact: England and Wales disagrees more (0.088%) than Scotland (0.020%) or Northern Ireland (0.015%). And the bulk is not flawless either: its own three counts fail to reconcile on 463 companies, and on 41 companies the pipeline sees more outstanding charges than the bulk records as ever having existed.

The decision

Preset A stays on `Mortgages.NumMortOutstanding = 0`. The change we had recommended would have taken it from 31,002 to 30,163, and:

That would delete real prospects to fix a phantom.

The record is also honest about what the evidence does not cover: charge age is measurable only for the LBG subset, because that is the only charge-date column in the file. For the other 543 lendered companies in preset A the live-versus-stale split is inferred from the same gradient rather than measured.

What this one taught us

Three things, and they are the reason this section is the longest in the document.

A number that looks like an error can be the data telling you about the world. 661 companies in a prospecting list with an incumbent attached is a bug-shaped fact, and it was not a bug.

Provenance beats plausibility. The two columns were compared on where they came from, how they were gated, and what they can and cannot know, rather than on which produced the more comfortable number.

A gap got recorded rather than papered over. Unlike the Gazette block and the contracts block, the lender block carries **no as-of date and no staleness flag**, so its currency cannot be checked from the file at all. The record calls that "a gap worth closing at source" rather than pretending the question was fully settled.

11. Making presets adjustable

The specification asked for one thing the first build did not deliver:

Presets populate the panel rather than bypassing it, so a user can see what a preset did and adjust it.

The first build made a preset an opaque server-side clause. It worked, and you could not see inside it or change it. That was logged as the one substantive deviation and closed on 21 August.

A preset now carries a second expression of itself in the panel's own vocabulary. Selecting one loads those conditions into the actual filter controls. The chips shown are read back off the live filter state rather than off a stored description, so they stay true after an edit.

One invariant keeps the two halves honest:

`where` remains the definition of a preset. **The count is always computed from `where`, never from the mapping**, so the two cannot drift silently in the direction that matters.

How it was verified, and why counts were not enough

The mapping was verified before any UI was written, **by SET DIFFERENCE rather than by count**. Two predicates can agree on a total and still select different companies.

For every preset, the companies matched by `where` and the companies matched by the filter expansion were compared in both directions. Eight of nine returned 0 and 0.

The ninth

`contract_no_borrowing` cannot be taken apart, and the set difference shows exactly how badly:

Preset	where	expansion	only in where	only in expansion
contract_no_borrowing	2,303	805,602	0	803,299

Its `contracts_won_12m > 0` condition has no filter control, because contracts are not a filterable source. Expressed without it the preset returns 805,602 companies instead of 2,303.

The decision was to leave it alone:

Inventing a contracts filter would change the filter system this document verified, so the preset stays exactly as it was: applied server-side from `where` , with its four conditions listed as fixed, dashed chips and the reason stated on screen.

The dashboard tells the user it cannot take this one apart, and why. That is better than a control that silently does something different from what its label says.

Two equivalences were also recorded here because they are not obvious, and both were measured at 0 rows of disagreement in each direction: `lifecycle = Trading` is exactly `is_active` , and `lbg = current` is exactly `n_lbg_charges_outstanding > 0` .

12. Scores, and what they are not

Four models, held to a presentation discipline that is stricter than the modelling:

Score	Horizon	Event	Measured hit rate at the top
Lending readiness	3 months	Takes on new secured borrowing	43 in 100
Credit risk	6 months	Hits a genuine insolvency event	16 in 100
Voluntary exit	6 months	Has a strike-off proposal filed	not ranked
Growth	12 months	Moves up a size tier	22 in 100

Three rules apply. Scores are shown as **bands, never as probabilities**, and the horizon is always visible. Scores exist only for companies whose status is exactly Active, 1,409,284 of 1,531,094, and for the rest the panel disappears with a reason rather than showing zeros.

Voluntary exit is deliberately not offered as a filter at all:

998 of its top 1,000 values are exact ties and the top-100 cutoff is shared by 139 companies. It cannot support a ranked or banded filter.

It still appears on the company page, where a band is a fact about one company rather than an invitation to sort a list by it.

13. The company profile and the timeline

The original design named the timeline as the thing that would justify the project:

One timeline with every signal from every source on it. This is the view no single branch can produce today, and it is the thing that will sell the project.

It survived intact, and it is the clearest expression of the whole architecture: a Gazette notice, a contract win and a property title in one column in date order, which no individual notebook can produce.

The rest of the profile follows the original list closely: header, a row of source tiles each showing an honest "nothing found", a panel per source, the model scores labelled as scores rather than predictions, and for every fact its source, date and confidence.

The confidence requirement came from a known problem, flagged before any code was written:

Matches are not always certain. Name plus postcode matching is about 92% correct, and only 55% of Gazette notices carry a company number at all. **The confidence value must survive all the way to the screen, so a shaky match never looks like a certain one.**

That is why the Evidence panel shows "certain match" against Gazette notices, and why grants and trade marks say "name match only".

14. Market Analytics

The Analytics page was added late and it works on a different principle from the rest of the dashboard. It presents Sneha's competitive market analysis, and the constraint was to change nothing about it.

The code comment records why the files are read where they sit:

The three CSVs are her saved outputs and are read where they sit rather than copied in: `serve.py` already reads from outside the repo for `data/`, and **duplicating a file is how two versions of the same number start to disagree.**

Eleven figures were rebuilt as native HTML, CSS and SVG rather than pasted in as notebook images, so they inherit the dashboard's typography and respond to the window. The numbers are not recomputed in the browser. A verification script compares every displayed figure against her CSVs and against an independent recomputation from the parquet, and it is part of the regression suite.

One deliberate constraint: **Analytics does not respond to the Home filters.** It always describes the whole market. A page that silently re-scoped itself would make it impossible to tell whether a number was about the market or about a filtered slice.

15. The interface

Two decisions here are worth recording because both were reversals.

The card system was unified late. The dashboard had grown two card languages: the company profile and landing page on a translucent glass treatment, Market Analytics on a later solid dark treatment with a green ambient glow. Side by side they read as two products. They were reconciled into one surface driven by a shared token set, with glass kept as the material and the analytics card contributing the geometry and the pointer response.

The liquid-glass treatment was scoped to controls, not everything. The refraction effect uses an SVG displacement filter inside `backdrop-filter`, which only Chromium resolves, so a capability probe sets an attribute on `<html>` and everything else degrades to plain blur. It is applied to navigation and controls and deliberately **not** to analytical cards, on the reasoning that a page where everything is glass has no hierarchy at all.

An attempt to build a full optical liquid-glass effect from a Three.js and GLSL reference was investigated and rejected on a specific technical ground rather than on effort. The shader samples its background from a WebGL render target, which works because everything behind the glass is also WebGL content. Behind our navigation is live DOM, and there is no supported way to rasterise live DOM into a WebGL texture. The workarounds all fail: `html2canvas` does not support `backdrop-filter`, which this dashboard uses on nearly every surface.

The animation on the view switcher was also reversed. A squash-and-stretch deformation was built, then removed in favour of a plain glide, on the grounds that the effect was decorating a control rather than helping anyone use it.

16. Defects found after the specification closed

The record in `FILTER_SPEC.md` ends on 21 August. Three more defects were found in the final verification and visual passes, and they are worth adding because two of them were invisible.

Two animations were frozen, not slow. The view marker and the KPI flip both used a `transform` transition on an unpromoted element, which makes it a main-thread animation. Clicking a view calls `refresh()`, which re-renders the result list and occupies the main thread for one to two seconds, and the animation sat behind that work. Measured: the marker's inline target was correct within 1ms but it did not visibly leave the old segment until the render finished. First movement went from *"never within 1.3s"* to *"1ms"* once the element was promoted with `will-change: transform`.

A stray `}` in the stylesheet had been silently eating a rule. The `html[data-field="light"]` block was closed with one brace too many, left over from when it sat inside a media query. Chromium recovers from a stray brace by consuming the next rule. For a long time the victim was the `@supports no-backdrop-filter` fallback, which therefore never worked. When the intro overlay was added in that position, the victim became the rule that made it a full-screen fixed curtain, so the loading video rendered as an unstyled transparent element with the whole dashboard drawing on top of it. Both rules came back when the brace was removed.

The logo was requesting files that never existed. `brand()` probed `assets/lloyds-logo.svg` and `assets/lloyds-logo.png` before falling through to the mark that is actually on disk, so **every page load logged two 404s**. Neither file was ever added; the naming convention existed only in a README. The probe list now names what exists, and the README records that the list is fetched with real requests so any wrong name costs a 404 on every load. The favicon was inlined as a data URI at the same time, and a normal page load now produces zero console errors.

17. What we can actually prove

The evidence, in the order it would convince someone:

- **Filtering:** 63 facet counts, 34 filter cases, 9 presets decomposed into 36 conditions, 22 hand-built combinations including 8 deliberate contradictions, 120 seeded random combinations. Zero count mismatches, zero rows failing the predicate they were selected by. Ground truth written independently from column semantics, not copied from the implementation.

- **Presets:** verified by set difference in both directions, not by count. Eight of nine return 0 and 0; the ninth is documented as a permanent exception with its exact divergence measured.
 - **Ordering:** 10 identical calls per view return identical rows; page walks of 300 to 600 slots show zero duplicates and zero skipped companies; four pages of 25 equal one call for 100.
 - **Market Analytics:** every displayed figure checked against Sneha's CSVs, the notebook output, and an independent recomputation from the parquet.
 - **Performance, measured rather than assumed:** a full panel repaint issues roughly 57 aggregate queries, one per option, at 0.58s cold. `/api/presets` recomputes all nine populations over 1.5M rows in 0.17s. The record notes plainly: *"DuckDB absorbs that without a cache. No optimisation is needed."*
-

18. Limitations that remain

Stated plainly, because most of them are recorded in the spec as open items rather than discovered later.

The lender block has no as-of date. Unlike the Gazette and contracts blocks it carries no staleness flag, so its currency cannot be checked from the file. This is the gap behind the whole of section 10, and it is a fix at source rather than in the dashboard.

Contracts cannot be filtered, which is why one preset can never be taken apart.

Three sources are name-matched, not number-matched: grants and trade marks on name plus postcode area, and the news search on cleaned name. Trade marks are the weakest at confidence 0.85 across all 15,713 rows, and the register itself is five years old.

News coverage is effectively nil. 467 searched, zero verified. That is a finding rather than a defect, but it means the news source contributes almost nothing to the dashboard today.

`data.js` is still 27 MB on disk purely so the server can lift a 20-key metadata block at startup.

The `gaz_new_in_july_flag` exclusion has not been applied anywhere, correctly, because the dashboard describes rather than predicts. It must be honoured before any `gaz_*` field is used as a model feature.

`/api/watchlist` still treats an unrecognised view as "gazette", the last silent fallback left in the API after the B3 fix. It is not reached by the UI.

There is no URL routing. Navigation never changes the address bar, so browser Back leaves the dashboard rather than returning to the previous view, and nothing is deep-linkable.

The data path was hardcoded until late in the project. It now resolves from `--data`, then the `LLoyds_DATA` environment variable, then a fallback, so the dashboard can run on a machine other than the one it was built on.

Sources used

Document	What it gave
<code>reports/integration-design.md</code>	The original problem statement, the signals-table plan, the three join rules, the master-list question, the known-problems list, the source ownership split including hiring
<code>dashboard/FILTER_SPEC.md</code>	The decision and error record. Sections 0 to 10: the redundancy rule, the filter taxonomy and NULL rule, preset definitions and restriction reasoning, the Gazette temporal decision, the five defects, the two-outstanding-measures investigation, the preset-adjustability work, the logo 404s
<code>dashboard/serve.py</code>	Source-integration decisions recorded as code comments: the property trustee threshold, the news false positive, trade mark vintage and confidence, the company-number cleaner check, why the market CSVs are read in place
<code>dashboard/data.js</code>	The evidence for the hiring source being dropped rather than completed, and the only surviving trace of the sharded build
The working record	The three-stage architecture history, the sharding design, and the DuckDB-against-PyArrow benchmark in section 2, none of which is in a repository file
The running dashboard	Confirmation that the described behaviour is the current behaviour

What could not be established from the records

The architecture history and the engine benchmark in section 2 are not in any file in the repository. The sharded `core.js` design and the DuckDB-against-PyArrow measurements come from the working record rather than from `FILTER_SPEC.md` or the code, and the intermediate sharded build no longer exists on disk to be inspected. `data.js` is the only surviving trace of it. The figures are reported as recorded.

One number in that record does not reconcile: the shard design is described as roughly 15,000 companies per shard, but 105,078 signalled companies across 100 shards is closer to 1,050 each. The shard sizes in KB and the 5 MB startup cost are consistent with the smaller figure, so the per-shard company count is left out of section 2 rather than guessed at.

The precise reason the hiring source produced no usable data is not recorded. What is recorded is that it was pending and never landed, and that it was removed rather than left as an empty tile.

The interface and liquid-glass decisions in sections 14 and 15 are documented only in code comments and in the working history, not in `FILTER_SPEC.md`, whose scope ends at the filter system. They are recorded here at a lower evidentiary standard than everything above them.